

1. Exploring Database

Setup MongoDB & Mongoose

1. **Asumsi:** sudah install MongoDB di laptop masing-masing.
2. Buka terminal lalu ketik: **mongo -version**
3. Jalankan MongoDB dengan mengetik: **mongod**.
4. Buka terminal satu lagi lalu ketik: **mongo**.
5. Ketik di terminal: **db** untuk melihat kita sedang berada di database apa.
6. By default kita terkoneksi dengan database **test**.
7. Buatlah database baru dengan mengetik: **use belajar_nodejs**.
8. Walaupun db belajar_nodejs **belum ada** tapi mongo akan memilihkan untuk kita.
9. Ketik: **show dbs**. Terlihat db belajar_nodejs belum ada. Database akan muncul di list jika sudah ada data yang dimasukkan.
10. Ketik: **show collections** untuk melihat collections yang ada di sebuah database. Collections disini sama seperti **table** di database MySQL.
11. Masukkan data ke db belajar_nodejs dengan mengetik: **db.coba.insert({"jurusan" : "informatika"})**
12. Sudah berhasil memasukkan satu item ke collection **coba**. Lalu ketik lagi: **show dbs**.
13. Ketik: **show collections**.
14. Sudah ada satu collection yaitu **coba**.
15. Untuk melihat data pada collection coba ketik: **db.coba.find().pretty()**
16. Buat collection baru dengan mengetik: **db.createCollection("app_chat")**.
17. Cek: **show collections**.
18. Sudah ada collection **app_chat**.
19. Untuk menghapus database ketik: **use <database_name>** lalu **db.dropDatabase()**.
20. Untuk menghapus collection ketik: **use <database_name>** lalu **db.<collection_name>.drop()**.
21. Agar dapat mengkoneksikan NodeJS ke MongoDB kita butuh **package Mongoose**.
22. Buka <http://mongoosejs.com/>.
23. Dengan object schemas pada mongoose akan membuat kita lebih mudah menggunakan MongoDB.
24. Buka terminal dan ketik **cd DemoApp**.
25. Jalankan aplikasi Node dengan mengetik: **nodemon .\server.js**. Demo sebentar aplikasi chat.
26. Install monggose dengan npm.
27. Buka terminal baru (click plus hijau) dan ketik: **cd DemoApp**
28. Lalu ketik: **npm install -s mongoose**.
29. Tambahkan code **require mongoose** berikut pada **server.js**:

```
var io = require('socket.io')(http);
var mongoose = require('mongoose');
```
30. Selanjutnya koneksikan aplikasi AppChat ke MongoDB.
31. Edit **server.js**:

```
var dbUrl = 'mongodb://localhost:27017/belajar_nodejs'

var server = http.listen(3000, function () {
  .....
});
```
32. Pindahkan **var server = http.listen()** ke baris paling bawah.
33. Koneksikan ke db mongoose dengan mengetik:

```
mongoose.connect(dbUrl).then(() => {
  console.log('koneksi ke Mongoddb berhasil!');
}).catch((err) => {
  console.error(err);
});

var server = http.listen(3000, function () {
  .....
});
```
34. Restart server. Lihat di terminal apakah muncul **koneksi mongoddb berhasil**.

35. Sekarang kita sudah terkoneksi ke database MongoDB.
36. Selanjutnya kita akan menyimpan data menggunakan mongoose.

Saving data with mongoose

1. Agar dapat menyimpan data menggunakan mongoose ada **struktur** yang harus di setup, yaitu model dan schema.

2. Buat message model di **server.js**:

```
var dbUrl = 'mongodb://localhost:27017/belajar_nodejs';

var Message = mongoose.model('Message', {
  nama: String,
  pesan: String
});
```

3. {nama: String, pesan: String} adalah **schema definition**.
4. Selanjutnya buat object dari model yang telah dibuat di atas.
5. Modifikasi **app.post()**:

```
app.post('/pesan', function (req, res) {
  var message = new Message(req.body);

  message.save().then(function(error) {
    if(error) {
      console.log(error)
    } else {
      pesan.push(req.body)
      io.emit('pesan', req.body);
      res.sendStatus(200)
    }
  })
})
```

6. Test di browser dengan memasukkan nama dan pesan lalu click tombol **Kirim**.
7. Data otomatis akan tersimpan di **collection messages**.
8. Query dokumen MongoDB dengan mengetik: **db.messages.find().pretty()**.
9. Hapus **pesan.push(req.body)** dari **app.post()**.
10. Lalu modifikasi **app.get()** menjadi:

```
app.get('/pesan', function (req, res) {
  Message.find({}).then((pesan) => {
    res.send(pesan)
  })
})
```

11. Hapus **array var pesan**.

```
var pesan = []
```

12. Refresh browser, pesan yang muncul berasal dari MongoDB.
13. Coba tes kirim pesan yang lain.
14. Langsung muncul pesan di browser karena menggunakan **socket.io** dan jika di refresh pesan masih tetap ada.
15. Coba hapus semua pesan di MongoDB dengan mengetik: **db.messages.remove({})**.
16. Refresh browser maka semua pesan hilang.